

Part A

1. Create an HTML form with fields for user's name, email, and message. Write a PHP script to handle form data submission and display the submitted information.

```
<form method="POST">
    <h1>Contact Form</h1>
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="mail" required><br>
    Message: <textarea name="msg" required></textarea>
    <input type="submit" name="submit">
</form>
<?php
    if(isset($_POST["submit"])){
        $name=$_POST["name"];
        $email=$_POST["mail"];
        $msg=$_POST["msg"];
        echo "<h3>Submitted Information</h3><br>Name: $name<br>Email:
$email<br>Message: $msg";
    }
?>
```

Output

Contact Form Name: GJatthan Email: alio@gmail.com Message: bdbcksdbcksdj <input type="submit" value="Submit"/>	Submitted Information Name: GJatthan Email: alio@gmail.com Message: bdbcksdbcksdj
--	---

2. Write a PHP program that collects input from the user through a textbox and checks whether the given number is an Armstrong number or not.
 - a. If the number is indeed an Armstrong number, display all numbers in range from 1 to that specified number (Note : 0 is Not an Armstrong Number).
 - b. Perform text validation, including checks for positive integers and general text input.

```
<h2>Armstrong number</h2>
<form method="post">
    Enter positive integer: <input type="number" name="number">
    <input type="submit" name="submit">
</form>

<?php
    function getArmstrong($num) {
        $sum=0;
        $totalDigits=strlen($num);
```

```

while ($num > 0) {
    $digit=(int)($num %10);
    $sum=$sum+pow($digit,$totalDigits);
    $num=(int)($num/10);
}
return $sum;
}
if(isset($_POST["submit"])){
    $num=$_POST["number"];
    if(empty($num))
        echo "<p>Please Enter a number</p>";
    else if($num<=0)
        echo "<p>Number must be positive and greater than 0</p>";
    else{
        if($num!=getArmstrong($num))
            echo "<p>$num is not armstrong</p>";
        else{
            echo "<p>$num is armstrong number</p><br><h2>Armstrong
numbers from 1 to $num</h2><br>";
            for( $i= 1; $i<=$num; $i++ )
                if($i==getArmstrong($i))
                    echo "$i ";
            }
        }
    }
}
?>

```

output

Armstrong number

Enter positive integer:

123 is not armstrong

Armstrong number

Enter positive integer:

153 is armstrong number

Armstrong numbers from 1 to 153

1 2 3 4 5 6 7 8 9 153

3. Write a PHP program demonstrating the usage of sessions to store and retrieve user credentials like username and password. Combine the login, welcome, and logout functionalities into two files – ‘index.php’ and ‘welcome.php’.

index.php

```
<?php
    session_start();
    if(isset($_SESSION["uname"])){
        header("Location: welcome.php");
        exit();
    }
    else if(isset($_POST["login"])){
        $_SESSION["uname"]=$_POST["uname"];
        $_SESSION["pass"]=$_POST["pass"];
        header("Location: welcome.php");
        exit();
    }
?>
<form method="post">
    <h1>Login Form</h1><br>
    Username : <input type="text" name="uname" required><br>
    Password: <input type="password" name="pass" required><br>
    <input type="submit" name="login" value="Login">
</form>
```

welcome.php

```
<?php
    session_start();
    if (!isset($_SESSION["uname"])){
        header("Location: index.php");
        exit();
    }
    if (isset($_POST["logout"])){
        session_destroy();
        header("Location: index.php");
        exit();
    }
    $uname=$_SESSION['uname'];
    $pass=$_SESSION['pass'];
    echo "Username: $uname<br>Password: $pass<br>";
?>
<form method="post">
    <input type="submit" name="logout" value="Logout">
</form>
```

output

Login Form

Username :

Password:

Username: Goutham

Password: 12346

4. Write a simple PHP program with a user interface for a mathematical calculator using HTML forms. Perform division by zero operation with appropriate display of messages. Perform text validation, special character validation for textboxes with appropriate display of messages

```
<h1>PHP Calculator</h1>
<form method="post">
  <input type="text" name="num1" placeholder="Enter first number"
required>
  <select name="operator">
    <option>+</option>
    <option>-</option>
    <option>*</option>
    <option>/</option>
  </select>
  <input type="text" name="num2" placeholder="Enter second number"
required>
  <input type="submit" value="Calculate" name="submit">

<?php
  if(isset($_POST["submit"])){
    $num1=$_POST["num1"];
    $num2=$_POST["num2"];
    $op=$_POST["operator"];
    $result="";
    if(!is_numeric( $num1 ) || !is_numeric( $num2 ) )
      $result= "Please enter a numeric value<br>";
    else{
      switch($op) {
        case "+": $result= $num1+$num2;
                  break;
        case "-": $result= $num1-$num2;
                  break;
        case "*": $result= $num1*$num2;
                  break;
        case "/": if($num2==0)
                    $result= "Division By Zero Error";
```

```

        else
            $result= $num1/$num2;
        break;
    }
}
echo $result;
}
?>
</form>

```

Output

PHP Calculator

10 + 20 Calculate 0.5

PHP Calculator

10 + 20 Calculate 30

PHP Calculator

10 / 0 Calculate Division By Zero Error

- Write a PHP program with a user interface for calculating age based on the user's birth date. Display the age in years, months, and days

```

<h1>Age Calculator</h1><br>
<form method="post">
    Enter your date of Birth: <input type="date" name="dob" required>
    <input type="submit" value="Calculate age" name="submit"><br>
</form>

<?php
if (isset($_POST["submit"])) {
    $dob = new DateTime($_POST["dob"]);
    $currentDate = new DateTime();
    if ($dob > $currentDate)
        echo "Please select valid date ";
    else {
        $age = $dob->diff($currentDate);
        echo "<b>Your age is $age->y years $age->m months $age->d
days<b>";
    }
}
?>

```

output

Age Calculator

Enter your date of Birth:

Your age is 2 years 2 months 29 days

6. Write a PHP program to create an associative array representing a dictionary with words as keys and their meanings as values. Allow users to input a word and display its meaning if found, otherwise, display a "Word not found" message. (Please keep 10 pairs of keys and their meanings)

```
<h1>Dictionary</h1><br>
<form method="POST">
    Enter a word: <input type="text" name="word">
    <input type="submit" name="submit">
</form>
<?php
if (isset($_POST["submit"])) {
    $dict = array(
        "apple" => "it is a fruit",
        "book" => "used to gain info",
        "mobile" => "used to communicate"
    );
    $word = strtolower($_POST["word"]);
    if (array_key_exists($word, $dict))
        echo "<b>Meaning</b>: $dict[$word]";
    else
        echo "$word not found in dictionary";
}
?>
```

Output

Dictionary

Enter a word:

Meaning: it is a fruit

Dictionary

Enter a word:

bmw not found in dictionary

7. Write a PHP program that includes a user form with a text field and submit buttons for various string manipulations. It will display the result accordingly. (for replace, replace 'a' with 'x').

```
<h1>String Manipulation</h1>
<form method="post">
    Enter a string:<input type="text" name="data" required>
    <br><br>
    <button name="action" value="length">Get Length</button>
    <button name="action" value="uppercase">Convert to Uppercase</button>
    <button name="action" value="lowercase">Convert to Lowercase</button>
    <button name="action" value="reverse">Reverse String</button>
    <button name="action" value="palindrome">Check Palindrome</button>
    <button name="action" value="shuffle">Shuffle Characters</button>
    <button name="action" value="count">word count</button>
    <button name="action" value="replace">Replace Characters</button>
</form>
<?php
if (isset($_POST["action"])) {
    $data = $_POST["data"];
    switch ($_POST["action"]) {
        case "length":
            echo "String Length: " . strlen($data);
            break;
        case "uppercase":
            echo "Uppercase: " . strtoupper($data);
            break;
        case "lowercase":
            echo "Lowercase: " . strtolower($data);
            break;
        case "reverse":
            echo "Reversed: " . strrev($data);
            break;
        case "palindrome":
            $lower=strtolower($data);
            if ($lower == strrev($lower))
                echo "The string is a palindrome.";
            else
                echo "The string is not a palindrome.";
            break;
        case "shuffle":
            echo "Shuffled: " . str_shuffle($data);
            break;
        case "count":
            echo "Word Count: " . str_word_count($data);
            break;
        case "replace":
            echo "After replacing 'a' with 'X': " . str_replace("a", "X",
$data);
            break;
    }
}
```

```

    }
}
?>

```

8. Write a PHP user interface program with an HTML form to input a string. Upon submission, it will display the number of times each word occurs, ignoring the distinction between capital and lowercase letters. It should also print the most and least used words. Additionally, include buttons for sorting data in ascending order and descending order.

```

<h1>Word Frequency Analyzer</h1>
<form method="post">
    Enter a string:<input type="text" name="data" required>
    <button name="action">Analyze</button>
</form>
<?php
    if (isset($_POST["action"])) {
        $data = strtolower($_POST["data"]);
        $words = str_word_count($data, 1);
        $frequency = array_count_values($words);

        echo "<h2>Word Frequencies:</h2>";
        foreach ($frequency as $word => $count)
            echo "$word: $count<br>";

        asort($frequency);
        $asort_arr=json_encode($frequency);
        echo "Least used word : ".key($frequency)." ".reset($frequency)."
times used<br>";
        arsort($frequency);
        $dsort_arr=json_encode($frequency);
        echo "Most used word : ".key($frequency)." ".reset($frequency)."
times used<br>";

        echo "<form method=post>
            <button name=sort value=$asort_arr>Sort Ascending</button>
            <button name=sort value=$dsort_arr>Sort Descending</button>
        </form>";
    }
    if(isset($_POST["sort"])) {
        $s_arr = json_decode($_POST["sort"], true);
        echo "<h2>Sorted Word Frequencies:</h2>";
        foreach ($s_arr as $word => $count)
            echo "$word: $count<br>";
    }
?>

```


Output

Word Frequency Analyzer

Enter a string:

Word Frequencies:

a: 6
b: 3
c: 1
d: 1
e: 1

Least used word : c 1 times used
Most used word : a 6 times used

Sorted Word Frequencies:

a: 6
b: 3
c: 1
d: 1
e: 1

Sorted Word Frequencies:

c: 1
d: 1
e: 1
b: 3
a: 6

Part B

1. PHP program to implement student registration form using Labels, Text Boxes, Text Area, Checkbox, Radio Buttons, Select and Submit button. (First Name, Last Name, Address, E-Mail, Mobile, City, State, Gender, Hobbies, Blood Group). Display user inserted value in a new PHP page in a neat format.

index.php

```
<h1>Student Registration Form</h1>
<form method="POST" action="B1Disp.php">
    <label>First Name:</label><input type="text" name="fname"
required><br><br>
    <label>Last Name:</label><input type="text" name="lname"
required><br><br>
    <label>Address:</label><textarea name="address"
required></textarea><br><br>
    <label>E-Mail:</label><input type="email" name="email"
required><br><br>
    <label>Mobile:</label><input type="tel" pattern="[6-9][0-9]{9}"
name="mobile" required><br><br>
    <label>City:</label><input type="text" name="city" required><br><br>
    State: <input type="text" name="state" required><br><br>
    Gender: <input type="radio" name="gender" value="Male" required> Male
    <input type="radio" name="gender" value="Female" required>
Female<br><br>
    Hobbies: <input type="checkbox" name="hobbies[]" value="Reading">
Reading
    <input type="checkbox" name="hobbies[]" value="Writing"> Writing
    <input type="checkbox" name="hobbies[]" value="Sports"> Sports<br><br>
```

```

Blood Group: <select name="blood_group" required>
    <option value="">Select</option>
    <option value="A+">A+</option>
    <option value="A-">A-</option>
    <option value="B+">B+</option>
    <option value="B-">B-</option>
    <option value="AB+">AB+</option>
    <option value="AB-">AB-</option>
    <option value="O+">O+</option>
    <option value="O-">O-</option>
</select><br><br>
<input type="submit" value="Submit">
</form>

```

B1Disp.php

```

<?php
    $hobbies = isset($_POST['hobbies']) ? implode(', ', $_POST['hobbies']) :
    'None';

    echo "<h1>Student Registration Details</h1><br>
        <b>First Name:</b>".$_POST['fname']. "<br>
        <b>Last Name:</b> ".$_POST['lname']. "<br>
        <b>Address:</b> ".$_POST['address']. " <br>
        <b>E-Mail:</b> ".$_POST['email']. " <br>
        <b>Mobile:</b> ".$_POST['mobile']. " <br>
        <b>City:</b> ".$_POST['city']. " <br>
        <b>State:</b> ".$_POST['state']. " <br>
        <b>Gender:</b> ".$_POST['gender']. " <br>
        <b>Hobbies:</b> $hobbies <br>
        <b>Blood Group:</b> ".$_POST['blood_group']. " <br>";
?>

```

Output

Student Registration Form

First Name:

Last Name:

Address:

E-Mail:

Mobile:

City:

State:

Gender: ☒ Male ☐ Female

Hobbies: ☒ Reading ☒ Writing ☐ Sports

Blood Group:

Student Registration Details

First Name: GJatthan
Last Name: GJatthan
Address: csdcsdvcds
E-Mail: grg@gmail.com
Mobile: 6868686868
City: Udupi
State: Karnataka
Gender: Male
Hobbies: Reading, Writing
Blood Group: B+

2. Develop a PHP program that facilitates the addition, multiplication of two matrices. Utilize HTML for the user interface and PHP for the backend logic. Dynamically generate the required number of textboxes based on the specified number of rows and columns. Implement three distinct buttons to perform each matrix operation

```
<form method="POST">
    Enter number of rows: <input type="number" name="rows" required><br><br>
    Enter number of columns: <input type="number" name="cols"
required><br><br>
    <input type="submit" name="generate" value="Generate Matrices">
</form>
<?php
    if (isset($_POST["generate"])) {
        $rows = $_POST["rows"];
        $cols = $_POST["cols"];

        echo "<form method=post>
            <input type=hidden name=rows value=$rows>
            <input type=hidden name=cols value=$cols>
            <h3>Matrix1</h3>
            <table border=1>";
            for ($i = 1; $i <= $rows; $i++) {
                echo "<tr>";
                for ($j = 1; $j <= $cols; $j++)
                    echo "<td><input type=number name=a$i$j
required></td>";
                echo "</tr>";
            }
            echo "</table><br>
            <h3>Matrix2</h3>
            <table border=1>";
            for ($i = 1; $i <= $rows; $i++) {
```

```

        echo "<tr>";
        for ($j = 1; $j <= $cols; $j++) {
            echo "<td><input type=number name=b$i$j
required></td>";
        }
        echo "</tr>";
    }
    echo "</table>
    <input type=submit name=action value=sum>
    <input type=submit name=action value=product><br><br>
    </form>";
}
if (isset($_POST["action"])) {
    $rows = $_POST["rows"];
    $cols = $_POST["cols"];
    switch ($_POST["action"]) {
        case "sum":
            echo "<h3>Sum of Matrices</h3>
            <table border='1'>";
            for ($i = 1; $i <= $rows; $i++) {
                echo "<tr>";
                for ($j = 1; $j <= $cols; $j++) {
                    $sum = $_POST["a$i$j"] +
$_POST["b$i$j"];

                    echo "<td>$sum</td>";
                }
                echo "</tr>";
            }
            echo "</table>";
            break;
        case "product":
            if($rows != $cols) {
                echo "Matrix multiplication is not possible for non-
square matrices.";
                break;
            }
            else {
                echo "<h3>Product of Matrices</h3><table border='1'>";
                for ($i = 1; $i <= $rows; $i++) {
                    echo "<tr>";
                    for ($j = 1; $j <= $cols; $j++) {
                        $result = 0;
                        for ($k = 1; $k <= $cols; $k++)
                            $result += $_POST["a$i$k"] *
$_POST["b$k$j"];

                        echo "<td>$result</td>";
                    }
                    echo "</tr>";
                }
            }
        }
    }
}

```

```

    }
    echo "</table>";
}
}
?>

```

Output

Enter number of rows:

Enter number of columns:

Matrix1

1	1
1	1

Matrix2

2	2
2	2

Sum of Matrices

3	3
3	3

Product of Matrices

4	4
4	4

Enter number of rows:

Enter number of columns:

Matrix1

2
2

Matrix2

1
1

Enter number of rows:

Enter number of columns:

Matrix multiplication is not possible for non-square matrices.

3. Write a PHP program that implements a class to add and find the difference of two distance values given in feet and inches. The user inputs are collected through an HTML form.

```

<form method="post">
    Enter distance 1 <br>
    feet: <input type="number" name="d1_feet" required><br>
    inches: <input type="number" name="d1_inches" required><br><br>
    Enter distance 2 <br>
    feet: <input type="number" name="d2_feet" required><br>
    inches: <input type="number" name="d2_inches" required><br><br>
    <input type="submit" name="action" value="Calculate">
</form>

```

```

<?php
class Distance {
    private $feet;
    private $inches;

    public function __construct($feet, $inches) {
        $this->feet = $feet;
        $this->inches = $inches;
    }

    public function add($other) {
        $totalInches = ($this->feet + $other->feet) * 12 + ($this->inches +
$other->inches);
        return new Distance(intval($totalInches / 12), $totalInches % 12);
    }

    public function difference($other) {
        $totalInches1 = ($this->feet * 12) + $this->inches;
        $totalInches2 = ($other->feet * 12) + $other->inches;
        $diffInches = abs($totalInches1 - $totalInches2);
        return new Distance(intval($diffInches / 12), $diffInches % 12);
    }

    public function __toString() {
        return "{$this->feet} feet and {$this->inches} inches";
    }
}
if (isset($_POST["action"])) {
    $distance1 = new Distance($_POST['d1_feet'], $_POST['d1_inches']);
    $distance2 = new Distance($_POST['d2_feet'], $_POST['d2_inches']);

    $sum = $distance1->add($distance2);
    $diff = $distance1->difference($distance2);
    echo "Sum: $sum <br> Difference: $diff";
}
?>

```

Output

Enter distance 1
feet:
inches:

Enter distance 2
feet:
inches:

Sum: 10 feet and 10 inches
Difference: 0 feet and 2 inches

4. Program to implement a login form where users enter their username and password. Validate the credentials against data stored in a MySQL database and grant access if they are correct.

Table Structure (usercred)

Field	Type
uname	varchar(10)
pass	varchar(10)

insert 2 record into it in php myadmin

```
<h1>Login Page</h1>
<form method="post">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <input type="submit" name="submit">
</form>
<?php
    if(isset($_POST["submit"])){
        $username = $_POST["username"];
        $password = $_POST["password"];
        try{
            mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
            $con=new mysqli("localhost","root","","phpdb");
            $res=$con->query("select * from usercred where
uname='$username' and pass='$password'");
            if($res->num_rows > 0)
                echo "Login successful. Welcome, $username!";
            else
                echo "Invalid username or password.";
        }
        catch(Exception $e){
            echo $e->getMessage();
        }
    }
    ?>
```

output

Login Page	Login Page
Username: densame	Username: Densame
Password: 	Password:
<input type="submit" value="Submit"/>	<input type="submit" value="Submit"/>
Login successful. Welcome, densame!	Invalid username or password.

5. Program to create a feedback form with fields for name, email, subject, and message. Store the submitted feedback data in a MySQL database for later review.

Table Structure (feedback)

Field	Type
name	varchar(100)
email	varchar(100)
subject	varchar(200)
message	text

```
<h2>Feedback Form</h2>
<form method="post">
    Name: <input type="text" name="name" required><br><br>
    Email: <input type="email" name="email" required><br><br>
    Subject: <input type="text" name="subject" required><br><br>
    Message:<br>
    <textarea name="message" rows="5" cols="30"
required></textarea><br><br>
    <input type="submit" name="submit" value="Submit Feedback">
</form>
<?php
    if (isset($_POST["submit"])) {
        $name = $_POST["name"];
        $email = $_POST["email"];
        $subject = $_POST["subject"];
        $message = $_POST["message"];

        try {
            $con = new mysqli("localhost", "root", "", "phpdb");
            $res = $con->query("insert into feedback values ('$name',
'$email', '$subject', '$message')");
            if ($res)
                echo "Feedback submitted successfully.";
            else
                echo "Error while submitting feedback.";
        } catch (Exception $e) {
            echo $e->getMessage();
        }
    }
?>
```

6. Develop a dynamic PHP application to efficiently manage and store customer information, encompassing key fields like Customer Number, Customer Name, Item Purchased, and Mobile Number in Database. This application should provide a userfriendly interface with strategically placed buttons to trigger specific functionalities. These functionalities include:

- Add Customer Information:** Clicking this button should dynamically reveal a form for entering new customer details. Include proper validation checks for mobile numbers (10 digits), and also for Customer id ensuring accuracy in data input.
- Delete Customer Records:** Triggering this button should prompt the appearance of a form, specifically requesting the Customer ID to identify and delete the corresponding customer record. And provide appropriate messages for incorrect inputs.
- Search for Particular Entries:** This function should unveil a search form when activated, allowing users to input Customer id to find specific customer records.
- Sort Database Based on Customer Id:** Clicking this button should facilitate the sorting of the entire database based on customer id.
- Display Complete Set of Records:** Activating this function should present a comprehensive display of all customer records.
- Interface Design:** Initially, the interface should only showcase functional buttons. Upon clicking a button, the respective form should dynamically appear, offering a tailored and focused user experience.
- Give proper messages after every transaction.

Table Structure (customers)

Field	Type
cid	int
cname	varchar(20)
item	varchar(20)
mobile	bigint

methods.php

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$con = new mysqli("localhost","root","","phpdb");

function addCustomer($cid, $cname, $item, $mobile){
    global $con;
    $sql = "INSERT INTO customers VALUES ($cid, '$cname', '$item', $mobile)";
    $con->query($sql);
    echo "Customer added successfully.";
}

function display($query){
    global $con;
    $result = $con->query($query);
    if($result->num_rows > 0){
        echo "<table border=1>
            <tr>
                <th>ID</th>
```

```

        <th>Name</th>
        <th>Item</th>
        <th>Mobile</th>
    </tr>";

    while($row=$result->fetch_array()){
        echo "<tr>
            <td>$row[0]</td>
            <td>$row[1]</td>
            <td>$row[2]</td>
            <td>$row[3]</td>
        </tr>";
    }
    echo "</table>";
} else {
    echo "No records found.";
}
}

function deleteCustomer($cid){
    global $con;
    $sql = "DELETE FROM customers WHERE cid=$cid";
    $con->query($sql);
    if($con->affected_rows > 0)
        echo "Customer deleted successfully.";
    else
        echo "$cid not found.";
}
?>

```

add.php

```

<form method="post">
    Customer ID:<input type="number" name="cid" required><br><br>
    Name:<input type="text" name="cname" required><br><br>
    Item:<input type="text" name="item" required><br><br>
    Mobile:<input type="number" name="mobile" required><br><br>
    <input type="submit" name="save" value="Add Customer">
    <a href="index.php">Back to Home</a>
</form>

<?php
if(isset($_POST["save"])) {
    try{
        require("methods.php");
        addCustomer($_POST["cid"], $_POST["cname"], $_POST["item"], $_POST["mobile"]);
    }catch(Exception $e) {
        echo $e;
    }
}
?>

```

delete.php

```
<form method="post">
    Enter Customer ID to delete: <input type="number" name="cid" required><br><br>
    <input type="submit" value="Delete" name="delete">
    <a href="index.php">Back to Home</a>
</form>
<?php
    if(isset($_POST["delete"])) {
        try{
            require("methods.php");
            deleteCustomer($_POST["cid"]);
        }catch(Exception $e){
            echo $e;
        }
    }
?>
```

search.php (Same as delete.php)

```
<form method="post">
    Enter Customer ID to search: <input type="number" name="cid" required><br><br>
    <input type="submit" value="Search" name="search">
    <a href="index.php">Back to Home</a>
</form>
<?php
    if(isset($_POST["search"])) {
        try{
            require("methods.php");
            $cid=$_POST["cid"];
            display("SELECT * FROM customers WHERE cid = $cid");
        }catch(Exception $e){
            echo $e;
        }
    }
?>
```

index.php

```
<h1>Customer Management</h1>
<form method="post">
    <button name="action" value="add.php">Add</button>
    <button name="action" value="delete.php">Delete</button>
    <button name="action" value="search.php">Search</button>
    <button name="action" value="sort">Sort</button>
    <button name="action" value="display">Display</button>
</form>
```

```

<?php
if (isset($_POST["action"])) {
    try {
        require("methods.php");
        $action = $_POST["action"];
        if ($action == "sort")
            display("SELECT * FROM customers ORDER BY cid");
        else if ($action == "display")
            display("SELECT * FROM customers");
        else
            header("Location: $action");
    } catch (Exception $e) {
        echo $e;
    }
}
?>

```

7. A PHP and MySQL programme that features a book shopping form that takes in the book number, book title, price, quantity, and a option to choose the book code. The bill with the discounted amount and net bill amount is then displayed. Additionally, bill data are stored in the table.

Table Structure (bills)

Field	Type
book_number	int
book_title	varchar(255)
price	decimal(10,2)
quantity	int
total_price	decimal(10,2)
discount_amount	decimal(10,2)
net_bill_amount	decimal(10,2)

```

<h1>Book Shopping</h1>
<pre>
    <form method="POST">
        Enter book number: <input type="number" name="books"
required><br><br>
        Enter book title: <input type="text" name="title"
required><br><br>
        Enter price: <input type="number" name="price" required><br><br>
        Enter quantity: <input type="number" name="quantity"
required><br><br>
        Book Code: <select name="code" required>
            <option value="0.15">101</option>
            <option value="0.20">102</option>
            <option value="0.25">103</option>
            <option value="0.05">Other</option>

```

```

        </select><br><br>
        <input type="submit" name="generate" value="Generate Bill">
    </form>
</pre>
<?php
if (isset($_POST["generate"])) {
    $bookNumber = $_POST["books"];
    $bookTitle = $_POST["title"];
    $price = $_POST["price"];
    $quantity = $_POST["quantity"];
    $disc_per = $_POST["code"];

    $totalPrice = $price * $quantity;
    $discountAmount = $totalPrice * floatval($disc_per);
    $netBillAmount = $totalPrice - $discountAmount;

    echo "<h2>Bill Details</h2><br>Book Number: $bookNumber<br>Book Title:
$bookTitle<br>Price: $price<br>
Quantity: $quantity<br>Total Price: $totalPrice<br>
Discount Amount: $discountAmount<br>Net Bill Amount: $netBillAmount<br>";

    try {
        mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
        $conn = new mysqli("localhost", "root", "", "phpdb");

        $sql = "INSERT INTO bills VALUES ($bookNumber, '$bookTitle', $price,
$quantity, $totalPrice, $discountAmount, $netBillAmount)";

        if ($conn->query($sql))
            echo "Bill data stored successfully.";
        else
            echo "Error";
    } catch (Exception $e) {
        echo $e;
    }
}
?>

```

8. Develop a web application for proficiently managing hotel reservations, employing PHP for backend logic and MySQL for data storage. The application should feature a well-structured database table encompassing essential fields such as Room Number (primary key), Room Type (e.g., single semi, single deluxe, double semi, double deluxe, dormitory), Capacity, and Status (booked or available). Within the application's user interface, provide a textbox for entering the room number and two distinct buttons for check-in and check-out functionalities. Ensure that both checkin and check-out operations are executed based on the entered room number, facilitating a seamless and intuitive user experience
 - a. Insert 5 records into the table through interface, reflecting both available and booked rooms.

- Lists all available rooms and booked rooms on the webpage.
- Change the booking status to "booked" when a user checks in.
- Change the room status to "available" when a user checks out.
- Displays appropriate messages for successful booking, check-out, or if the room number is not present or not in the expected status

Table Structure (hotel)

Field	Type	Null	Key
roomNo	int	NO	PRI
roomType	varchar(50)	NO	
capacity	int	NO	
status	varchar(50)	NO	

methods.php

```
<?php
```

```
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$con = new mysqli("localhost","root","","phpdb");

function addHotel($roomNo,$roomType,$capacity){
    global $con;
    $sql = "INSERT INTO hotel VALUES($roomNo, '$roomType', $capacity,'Available')";
    $con->query($sql);
    echo "Hotel added successfully.";
}

function display($status){
    global $con;
    $result = $con->query("SELECT * FROM hotel WHERE status='$status'");
    if($result->num_rows > 0){
        echo "<table border=1><tr><th>Room No</th><th>Room
Type</th><th>Capacity</th></tr>";
        while($row=$result->fetch_array())
            echo "<tr><td>$row[0]</td><td>$row[1]</td><td>$row[2]</td></tr>";
        echo "</table>";
    } else
        echo "No records found.";
}

function updateStatus($roomNo, $status){
    global $con;
    $sql = "UPDATE hotel SET status='$status' WHERE roomNo=$roomNo";
    $con->query($sql);
    if($con->affected_rows > 0)
        echo "Status updated successfully.";
    else
        echo "$roomNo not found.";
}
```

```

function getStatus($roomNo){
    global $con;
    $sql="select status from hotel where roomNo=$roomNo";
    $result=$con->query($sql);
    if($result->num_rows>0){
        $result=$result->fetch_array();
        return $result[0];
    }
    return "Invalid";
}
?>

```

index.php

```

<form method="post">
    <h1>Add Room Details</h1>
    Room No:<input type="number" name="roomNo"><br><br>
    Room Type:<select name="roomType">
        <option value="Single">Single</option>
        <option value="Double">Double</option>
        <option value="Semi">Semi</option>
        <option value="Deluxe">Deluxe</option>
        <option value="Dormitory">Dormitory</option>
    </select><br><br>
    Capacity:<input type="number" name="capacity"><br><br>
    <button name="action" value="add">Add Room</button><br><br>
    <h1>Check In Check Out</h1>
    Enter room number: <input type="text" name="rNo">
    <button name="action" value="Booked">CheckIn</button><br><br>
    <button name="action" value="Available">CheckOut</button>
</form>
<?php
try {
    require("method.php");
    if (isset($_POST["action"])){
        $action = $_POST["action"];
        if($action=="add"){
            echo $_POST["roomNo"];
            addHotel($_POST["roomNo"], $_POST["roomType"], $_POST["capacity"]);
        }
        else{
            $currentStatus=getStatus($_POST['rNo']);
            if($currentStatus==$action)
                echo "Room is already $action.";
            elseif($currentStatus=="Invalid")

```

```
        echo "Invalid room number.";
    else{
        updateStatus($_POST['rNo'], $action);
        echo "<br>Room status updated to $action.";
    }
}
}
echo "<h2>Current Available Room Details</h2>";
display("Available");
echo "<h2>Current booked Room Details</h2>";
display("Booked");
} catch (Exception $e) {
    if ($e->getCode() == 1062)
        echo " Room already exists!";
    else
        echo $e;
}
?>
```